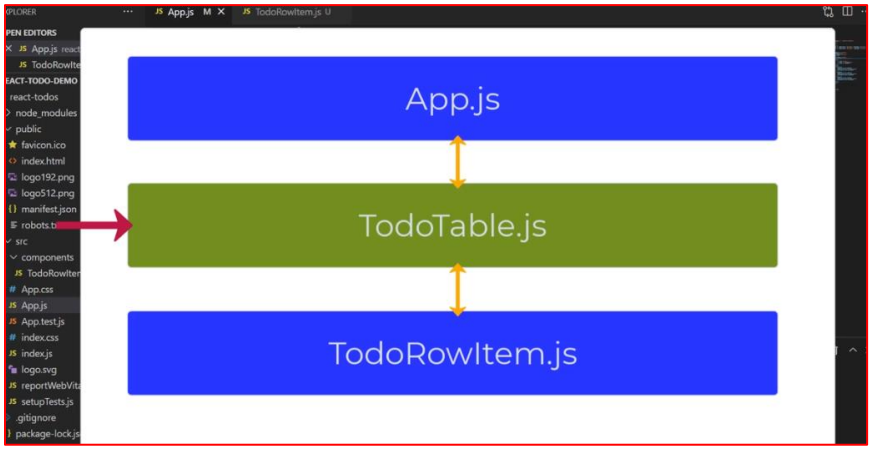
Create a Second React Component

Continuing with our development process for our to-do application, we now have our "ToDoRowItem" being completely dynamic. This dynamism is achieved by passing in the "props" property, and subsequently calling "props.rowNumber," "props.rowDescription," and "props.rowAssigned."

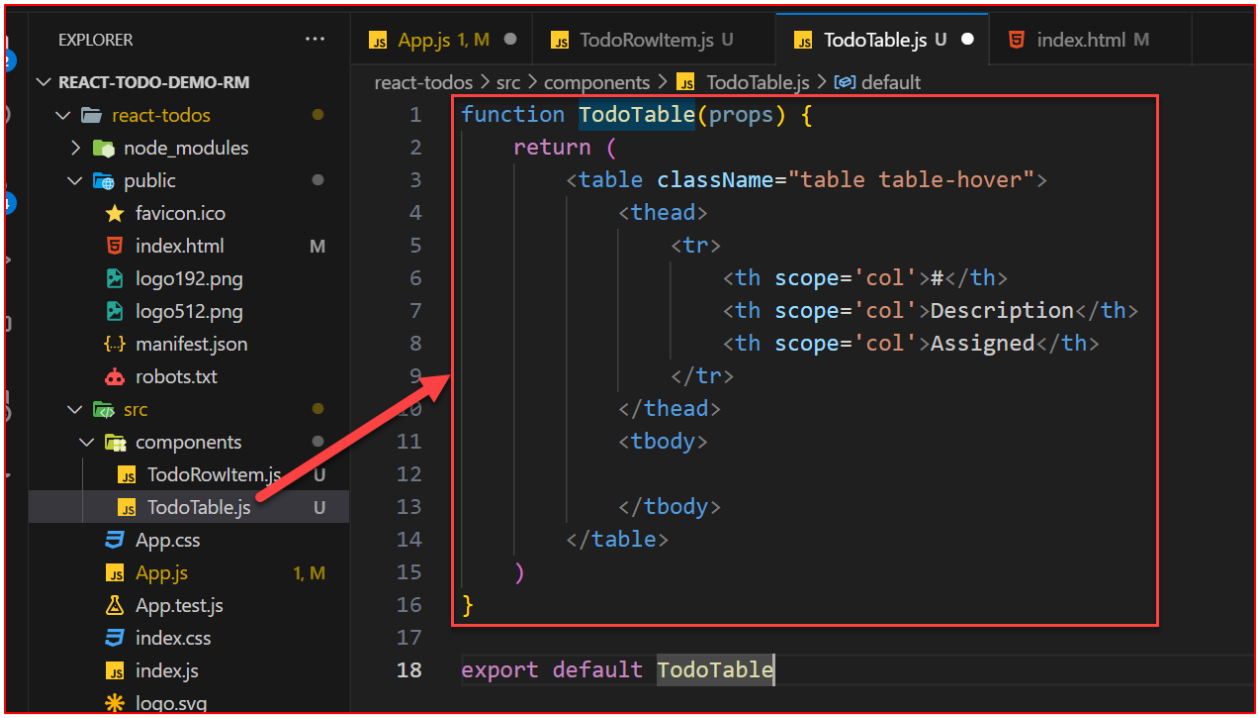
Now, an interesting point is that our entire table element could be a reusable component. This would allow us to organize it in a separate file for better management. With this in mind, we'll create a new component called "ToDoTable." This component will call our "ToDoRowItem" child component, rendering everything back up the hierarchy to "App.js," thus maintaining our to-do application.



Creating the "ToDoTable" Component

First, let's create a new file called "ToDoTable.js." Inside this file, we'll render a table similar to what we currently have. Here's the step-by-step process:

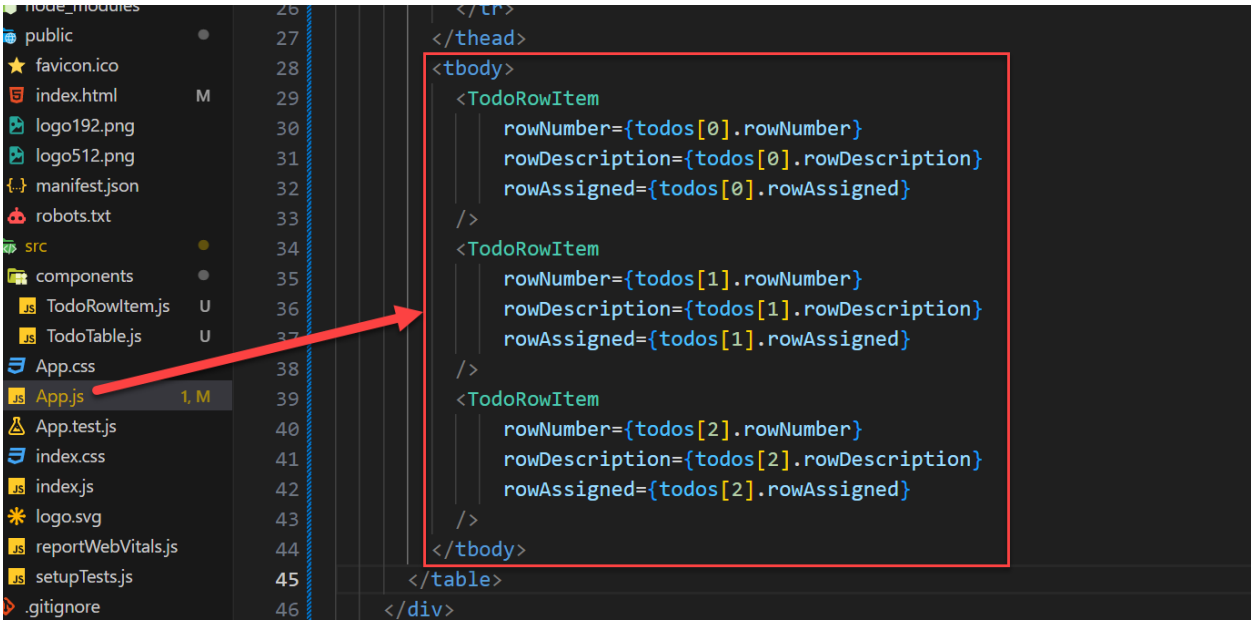
1. **Create a Function**: We'll create a new function called "ToDoTable," passing in "props" for now, even though we haven't called this component yet.
2. **Return the Table**: We will return a table with class name "table table-hover," along with table headings for "Pound Sign," "Description," and "Assigned."
3. **Export the Component**: Finally, we'll export the component with `export default ToDoTable.`



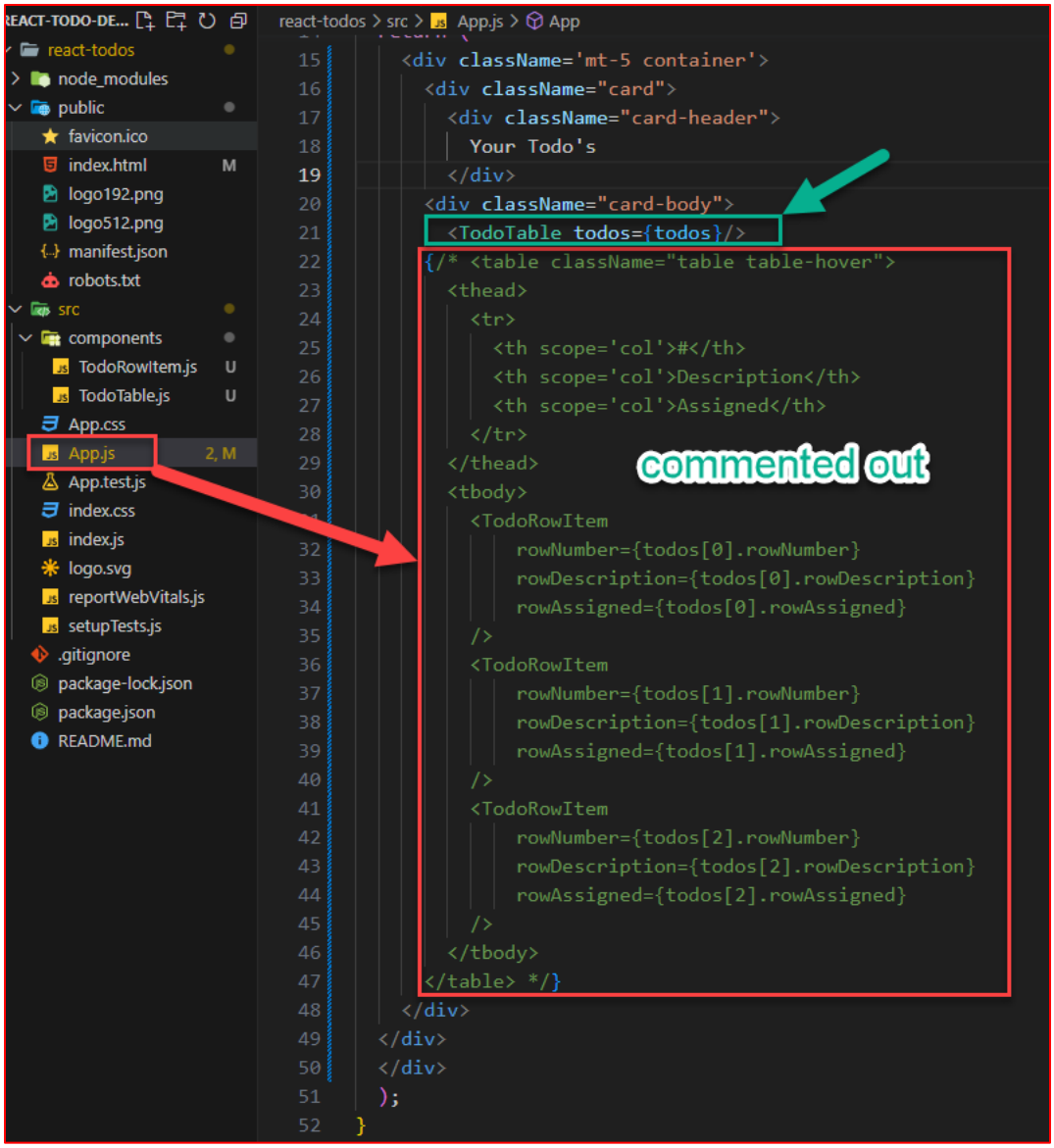
Implementing "ToDoTable" in App.js

Let's now shift back to our "App.js" file:

1. **Move Row Items**: Cut the three "ToDoRowItem" elements from "App.js" and paste them inside the body of "ToDoTable."

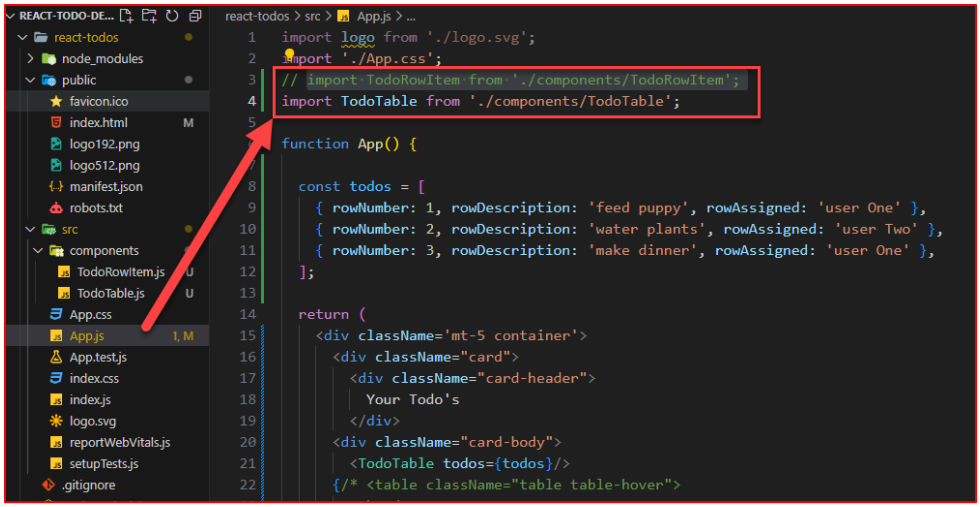


2. **Replace Old Table**: Delete the current table contents and call our new "ToDoTable" component.




```
    <div className='mt-5 container'>
      <div className="card">
        <div className="card-header">
          Your Todo's
        </div>
      <div className="card-body">
        <TodoTable todos={todos}/>
      {/* <table className="table table-hover">
        <thead>
          <tr>
            <th scope='col'>#</th>
            <th scope='col'>Description</th>
            <th scope='col'>Assigned</th>
          </tr>
        </thead>
        <tbody>
          <TodoRowItem
              rowNumber={todos[0].rowNumber}
              rowDescription={todos[0].rowDescription}
              rowAssigned={todos[0].rowAssigned}
          />
          <TodoRowItem
              rowNumber={todos[1].rowNumber}
              rowDescription={todos[1].rowDescription}
              rowAssigned={todos[1].rowAssigned}
          />
          <TodoRowItem
              rowNumber={todos[2].rowNumber}
              rowDescription={todos[2].rowDescription}
              rowAssigned={todos[2].rowAssigned}
          />
        </tbody>
      </table> */}
      </div>
    </div>
    </div>
    );
  }
```

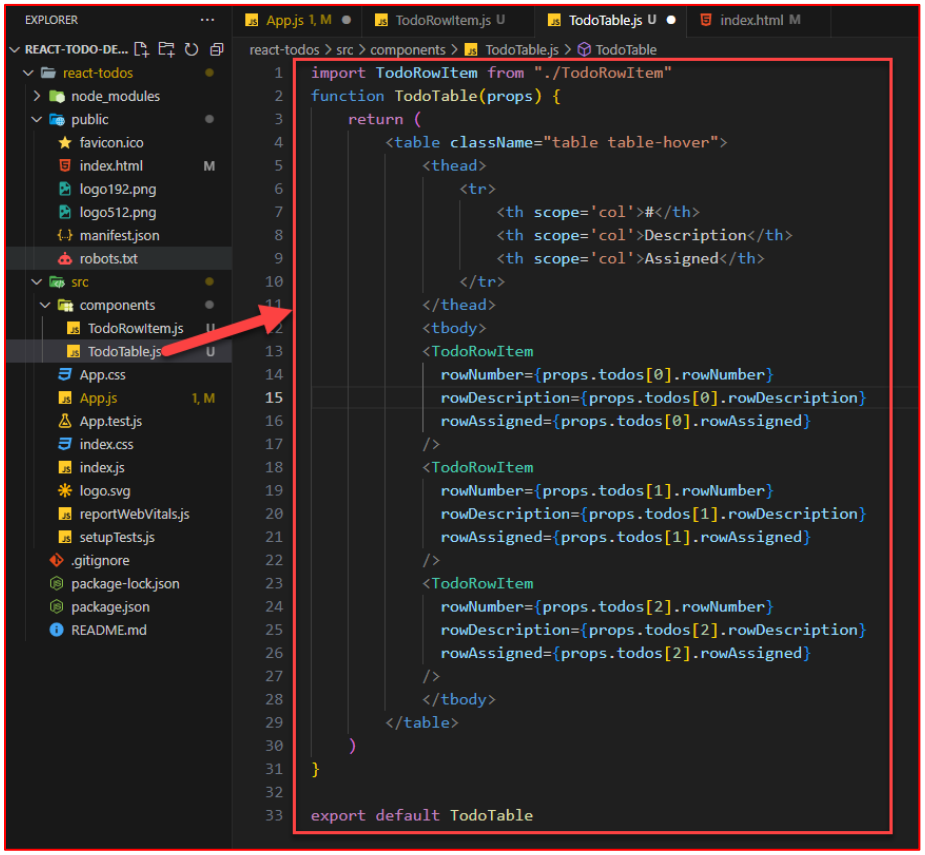
3. **Pass the Props**: The "ToDoTable" needs to take in a prop, so we'll say `todos={todos}`, passing in the "todos" variable from "App.js."
4. **Import the Component**: If it's not automatically imported, manually add `import ToDoTable from './components/ToDoTable';`.
5. **Remove Unused Import**: Since "ToDoRowItem" is no longer used in "App.js," we can remove that import.



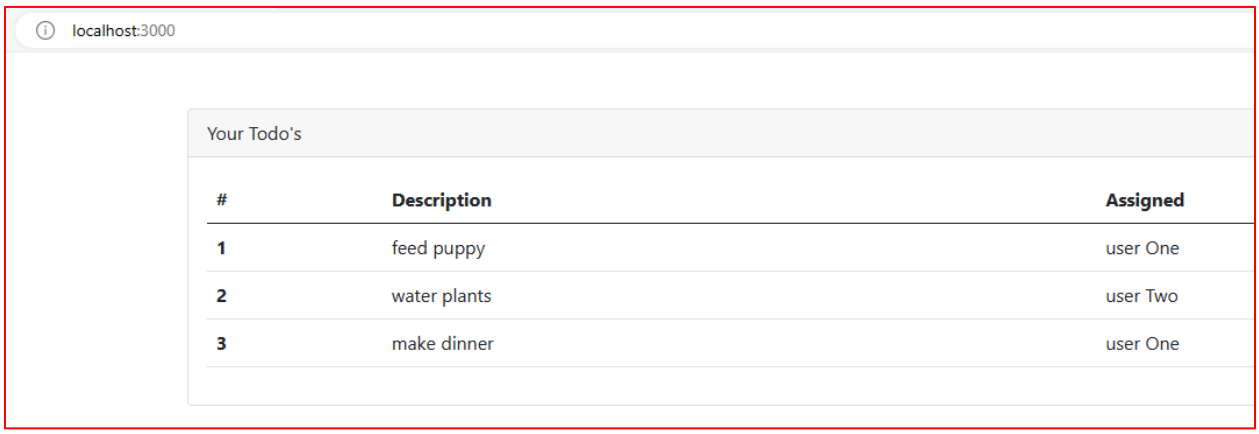
Completing the ToDoTable Component

Back in "ToDoTable.js":

1. **Import ToDoRowItem**: We need to import "ToDoRowItem" using `import ToDoRowItem from './ToDoRowItem';`.
2. **Use Props for ToDoRowItem**: Since the "todos" are getting passed in as a prop, we can use `props.todos` for all three of these row items.



After making these changes, refresh the page, and you'll see that the application works just like before.



Recap

In this lesson, we:

- Created a new component, "ToDoTable," to return an entire HTML table.
- Nested "ToDoRowItem" three times inside "ToDoTable."
- Utilized the "props" variable to pass data between components.

We've essentially created a hierarchy of components within our to-do application, fostering better organization and code reusability.